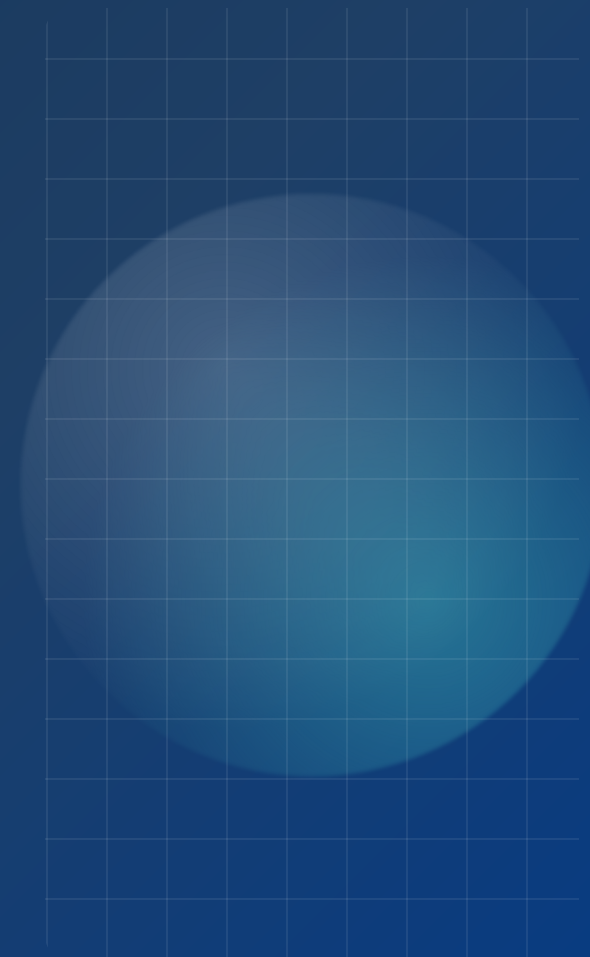


Neo4j as a selectable graph DB engine via Bolt

End-to-end demo on fevm-mjolnir with a real PFAS research-paper ontology — 303 triples written into Neo4j over Bolt, 99 inferred via OWL 2 RL. Post-review iteration ships secret-resource auth, Cypher INFO logging, a modular split, and the wired Test-connection flow.



v0.6 Neo4j slot — landed, end-to-end

OntoBricks now supports **Neo4j (Bolt / Cypher)** as a selectable Graph DB engine alongside Lakebase Postgres. Opt-in via Settings → Triple Store → Global → Neo4j (Bolt). Lakebase remains the default; no breaking changes.

32

classes generated from your PFAS paper by the AI ontology generator

303

RDF triples materialized into Neo4j by the OntoBricks build pipeline

99

inferred triples from T-Box OWL 2 RL reasoning, in 0.102 s

10.3 s

full build time: prepare → view → graph

- **C1 / C2 preserved** — Writes only via the OntoBricks build pipeline. `Neo4jStore.execute_query` raises `NotImplementedError` by design — no raw Cypher entry point.
- **Cleanup done** — `WaterTreatment_V1` label dropped from the demo Neo4j after capture. Total nodes: **0**. Paper stays in `fevm-mjolnir`.
- **v0.7 post-review iteration (2026-06-25)** — 6 commits address every point of Benoit's 2026-06-18 review: secret-resource auth, Cypher INFO logging, modular split (4 files), Settings UI flash fix, version bump to 0.7.0, and a wired Test-connection flow.

Why now

3 / 27

Benoit's v0.5 introduced the GraphDBBackend abstraction. The v0.6 roadmap is to land Neo4j as the first non-Lakebase backend, originally targeted at August 2026. This PR delivers it ahead of schedule with a complete user-visible path: pick the engine in Settings, build, query.

C1 OntoBricks controls the graph

Single canonical source of truth = the OntoBricks build pipeline. Mirror modes, dual-writes, etc., were explicitly ruled out in the 2026-05-20 sync.

C2 The ontology controls writes

No raw Cypher entry point exposed to users.
`execute_query` raises `NotImplementedError`
— all writes go through validated R2RML.

C3 No raw data in the graph

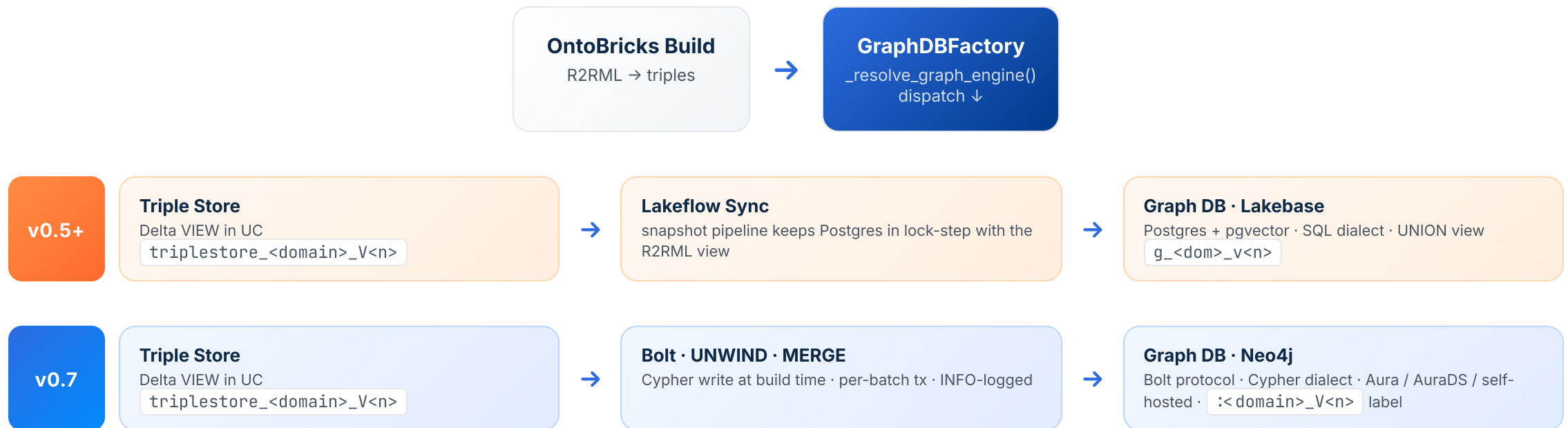
Only ontology-shaped triples land in Neo4j. The synthetic UC fact table stays in Unity Catalog; Neo4j receives the resolved entities.

- **Customer value** — Customers already on Neo4j (Ryan's contacts, EU manufacturing) get an OntoBricks deployment without forcing Lakebase.
- **Architectural credibility** — Proves the GraphDBBackend abstraction is real and ships with two backends, not one and a placeholder.

Factory dispatch, two backends

4 / 27

Engine selection is resolved once via `TripleStoreFactory._resolve_graph_engine` against `GlobalConfigService`. The factory dispatches to either `LakebaseStore` (existing) or `Neo4jStore` (this PR). **Both backends share the same 3-card arch shown in the Build / Cockpit UI** — Triple Store → Sync mechanism → Graph DB — they differ only in what fills each card.



- **Same GraphDBBackend interface** on both rows. The Triple Store card is identical (Delta view, R2RML-built). Only the middle card (sync mechanism) and the Graph DB card change.

■ OntoBricks v0.6 · PR #47

- Capability flags on `Neo4jStore`: `supports_cypher=True`, `query_dialect="cypher"`, `supports_graph_model=False` (flat triple in v1).

2026-06-12

What's in the PR — backend

FILE	PURPOSE
<code>back/core/graphdb/neo4j/Neo4jStore.py</code> NEW · ~580 LOC	Full <code>GraphDBBackend</code> impl. 16 named-query methods in native Cypher. <code>execute_query</code> raises <code>NotImplementedError</code> (C2).
<code>back/core/graphdb/GraphDBFactory.py</code>	<code>_create_neo4j</code> dispatch + <code>NEO4J_AVAILABLE</code> guarded import.
<code>back/objects/session/GlobalConfigService.py</code>	<code>ALLOWED_GRAPH_ENGINES</code> = ("lakebase", "neo4j") .
<code>back/core/reasoning/SWRLFlatCypherTranslator.py</code> NEW	Scaffolded mirror of SQL translator. Returns <code>None</code> + logs warning. Reasoning reports 0 violations / 0 inferences gracefully until the full translator lands.
<code>pyproject.toml</code>	<code>[project.optional-dependencies]</code> <code>neo4j</code> = ["neo4j ≥ 5.0"] .
<code>app.yaml.template</code>	Adds <code>--extra neo4j</code> so the deployed App ships the driver.

What's in the PR — frontend

6 / 27

FILE	PURPOSE
front/templates/settings.html	<code><option value="neo4j">Neo4j (Bolt)</option></code> + Neo4j config form (URI, db, auth, creds, encrypted).
front/config/menu_config.json	"Neo4j" item in the Settings sidebar.
front/static/config/js/settings.js	Save/load wiring for the Neo4j engine config.
front/static/domain/js/domain-validation.js	Pre-Build, <code>async-fetch /settings/graph-engine</code> so Validation card title is engine-aware.
front/static/query/js/query-sync.js	Same engine-aware fallback for the Build page title.
front/fastapi/dependencies.py ROOT CAUSE FIX	<code>triplestore_page_context</code> previously hard-coerced every engine to <code>"lakebase"</code> (<code>_raw if _raw == "lakebase"</code> else <code>"lakebase"</code> — a tautology). Now passes the resolved engine through directly.

The server-side fix + the two JS fetch fallbacks are defence in depth: page works correctly even on a stale `__TRIPLESTORE_CONFIG.graph_engine`.

The path — paper to triples

7 / 27

Real research paper goes in, real RDF triples come out the other side of the OntoBricks build pipeline, written into Neo4j over Bolt via the new Neo4jStore. Synthetic 12-row UC fact table provides the ABox instances — the only fake bit; numbers are illustrative.

1 Settings

Switch global engine to Neo4j (Bolt). Fill Bolt URI + creds + encrypted on.

2 Documents + Generate

Upload `AV-TR-2026-001_MMSF-PFAS-Risk-Assessment.pdf`. AI generates a 32-class ontology.

3 Data Source

Create + load

`mjolnir_catalog.ontobricks.pfas_measurements_demo`
(12 rows, paper-aligned schema).

4 Auto-Map

Batch run maps 25/25 entities and 12/12 relationships. 7 abstract entities excluded (no anchor column).

5 Diagnostics + Build

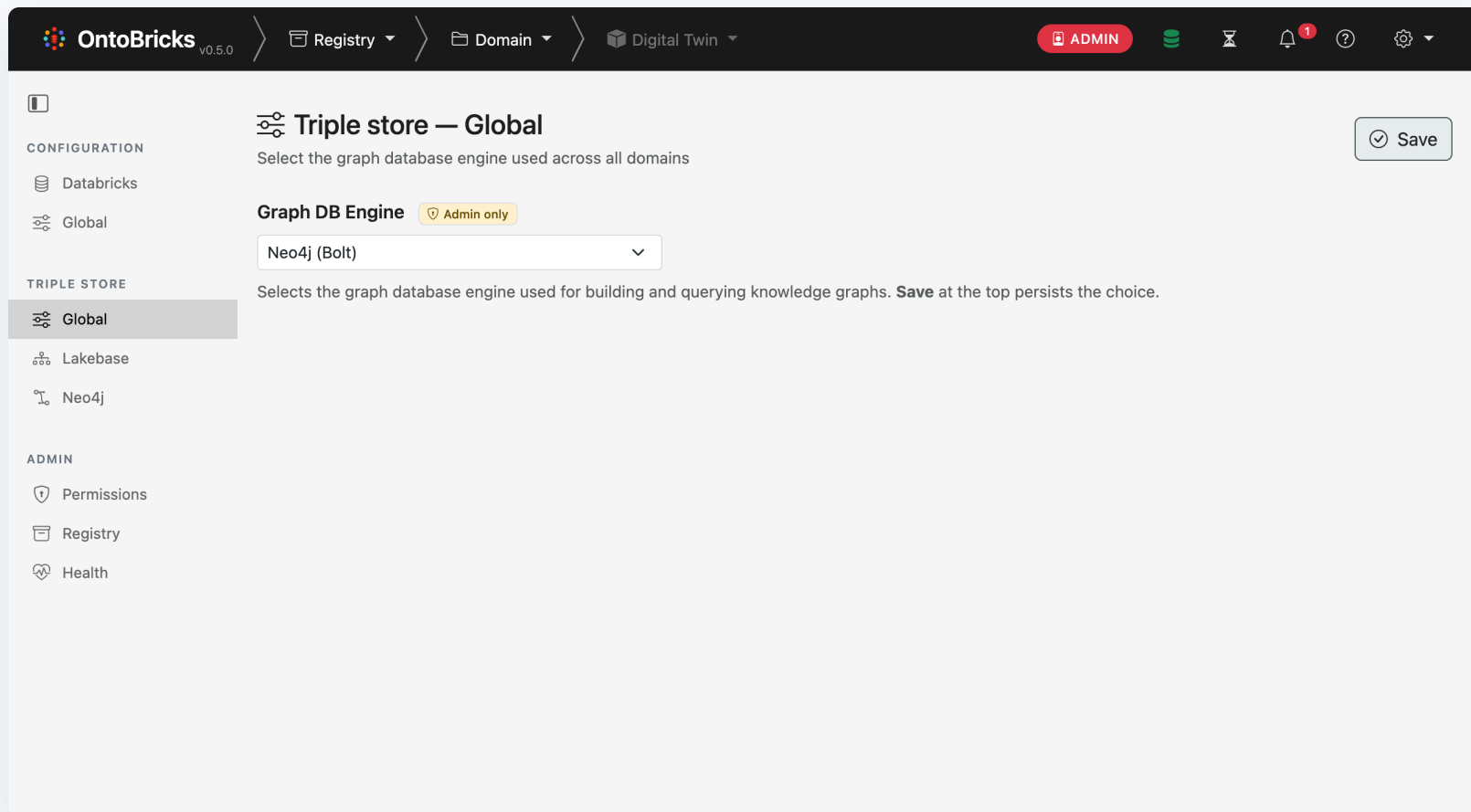
0 errors. Build pipeline emits 303 triples in 10.3 s into the new Neo4j backend.

6 Verify + Inference

Neo4j Browser shows 303 nodes. T-Box reasoning produces 99 inferred triples in 0.1 s.

Switch engine to Neo4j

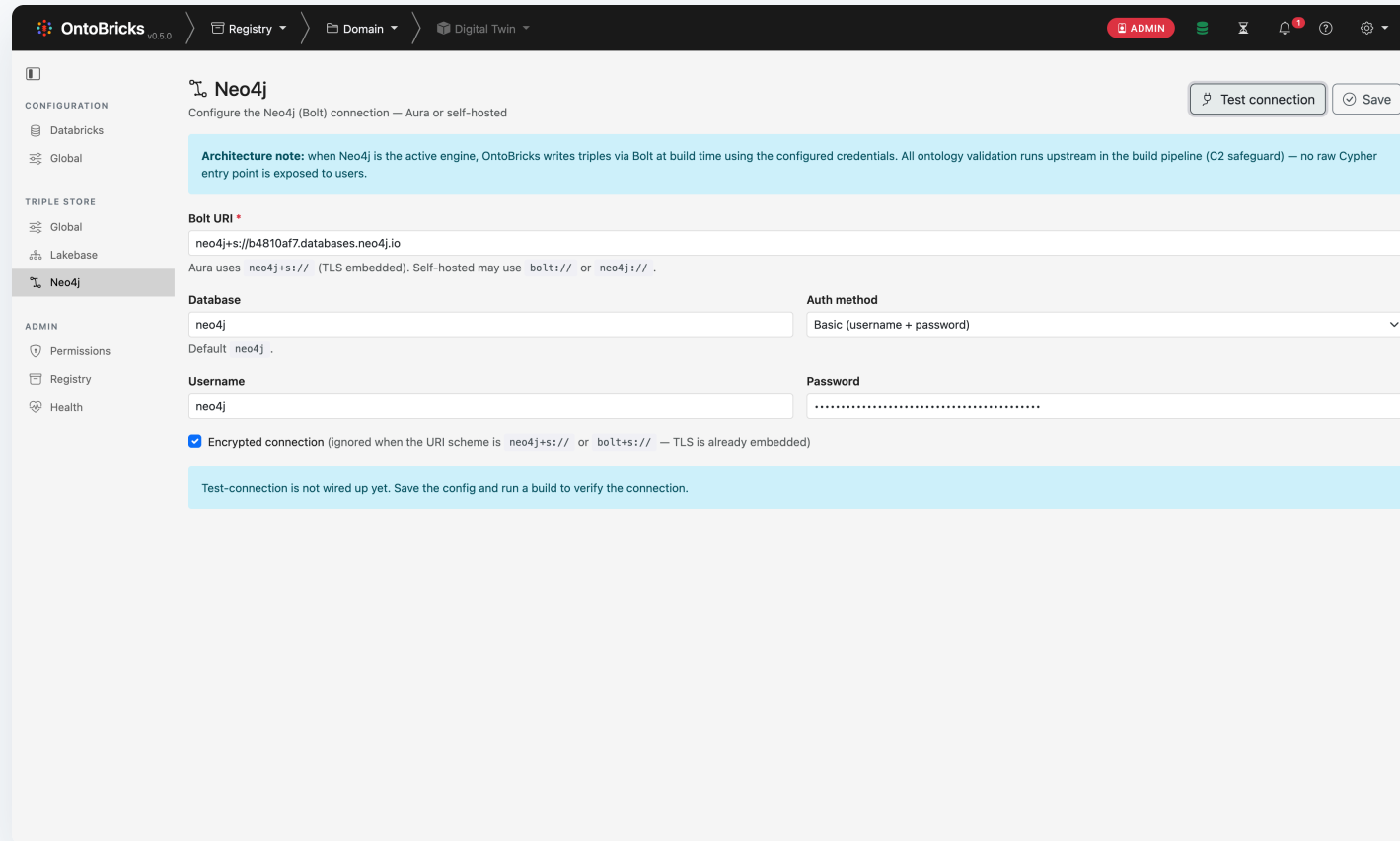
8 / 27



Triple Store → Global → **Neo4j (Bolt)** · saved confirmation toast. 01-settings-global-neo4j-saved.png

Neo4j connection form

9 / 27



OntoBricks v0.5.0 > Registry > Domain > Digital Twin > ADMIN

Neo4j

Configure the Neo4j (Bolt) connection — Aura or self-hosted

[Test connection](#) [Save](#)

Architecture note: when Neo4j is the active engine, OntoBricks writes triples via Bolt at build time using the configured credentials. All ontology validation runs upstream in the build pipeline (C2 safeguard) — no raw Cypher entry point is exposed to users.

Bolt URI *

neo4j+s://b4810af7.databases.neo4j.io

Aura uses neo4j+s:// (TLS embedded). Self-hosted may use bolt:// or neo4j:// .

Database

neo4j

Default: neo4j .

Auth method

Basic (username + password)

Username

neo4j

Password

.....

☒ Encrypted connection (ignored when the URI scheme is neo4j+s:// or bolt+s:// — TLS is already embedded)

Test-connection is not wired up yet. Save the config and run a build to verify the connection.

Bolt URI · basic auth · encrypted on. Form is engine-generic — works against any Bolt endpoint (Aura, AuraDS, self-hosted, on-prem). Test-connection is a stub for now — "Save & run a build to verify".

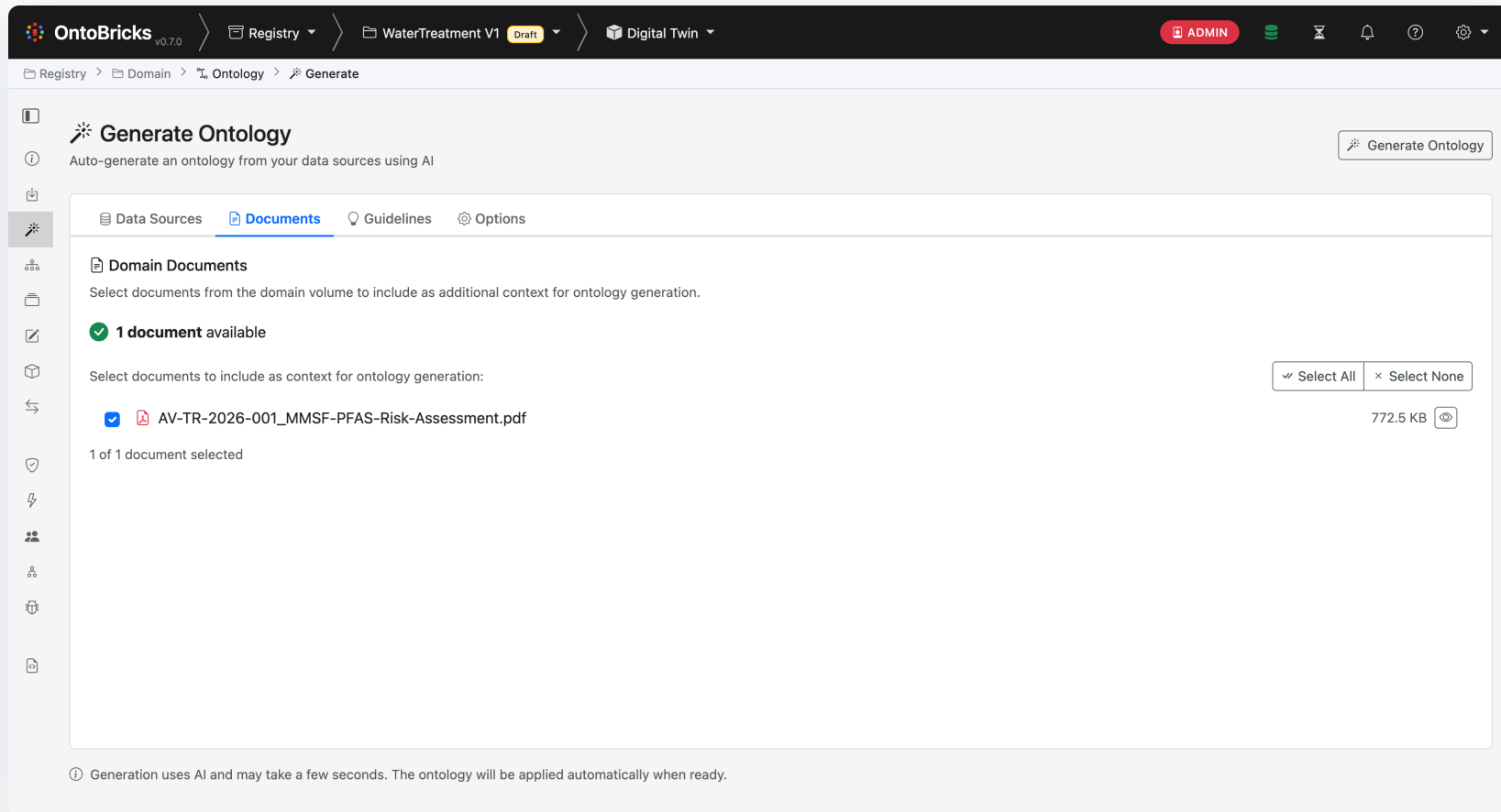
■ **OntoBricks v0.6 · PR #47**

02-settings-neo4j-form-filled.png

2026-06-12

PDF in, ontology out

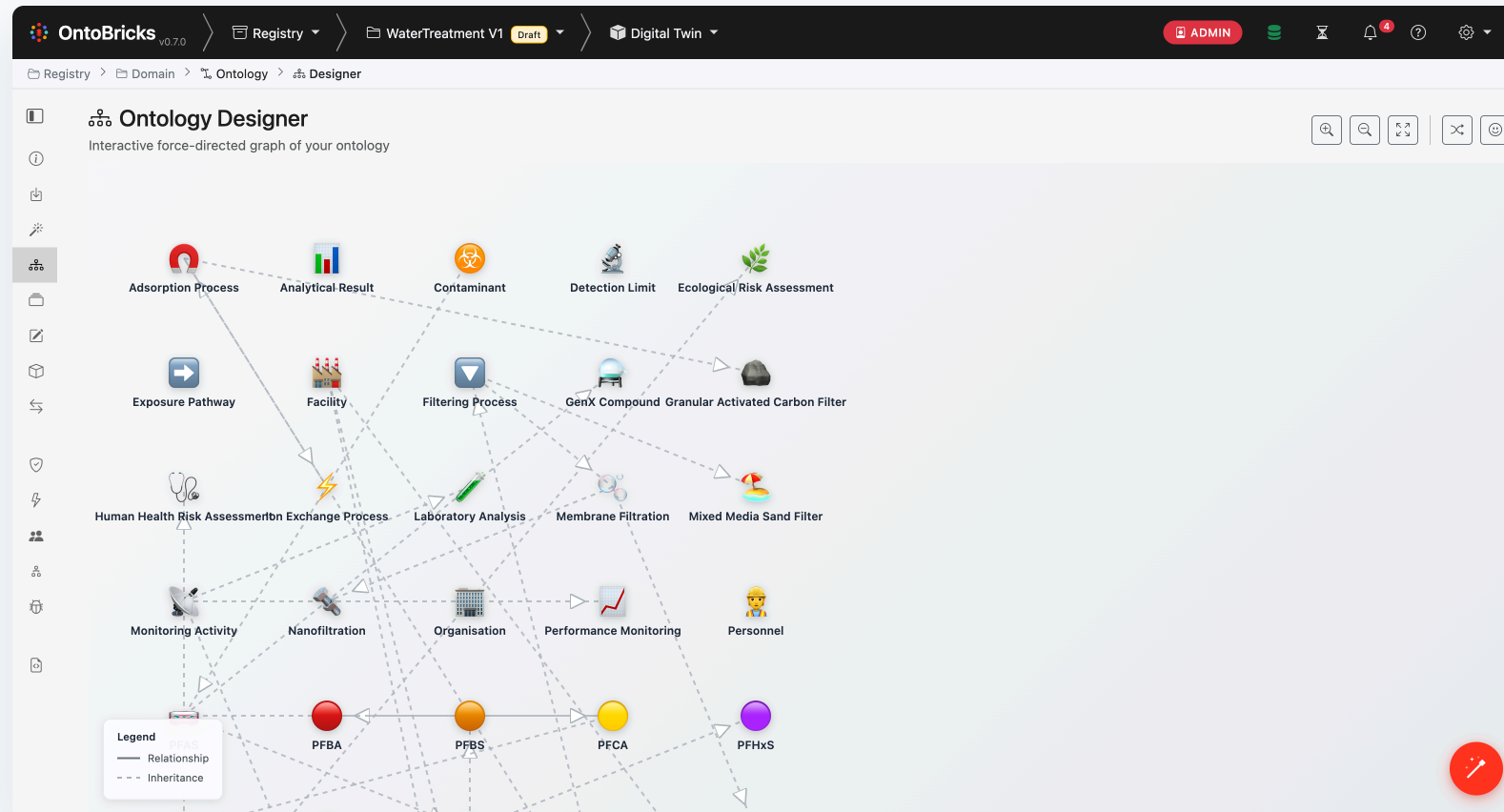
10 / 27



Generate Ontology → Documents tab · PFAS Risk Assessment PDF selected as AI context. 03-ontology-generate-pdf-selected.png

32 classes, 13 relations

11 / 27



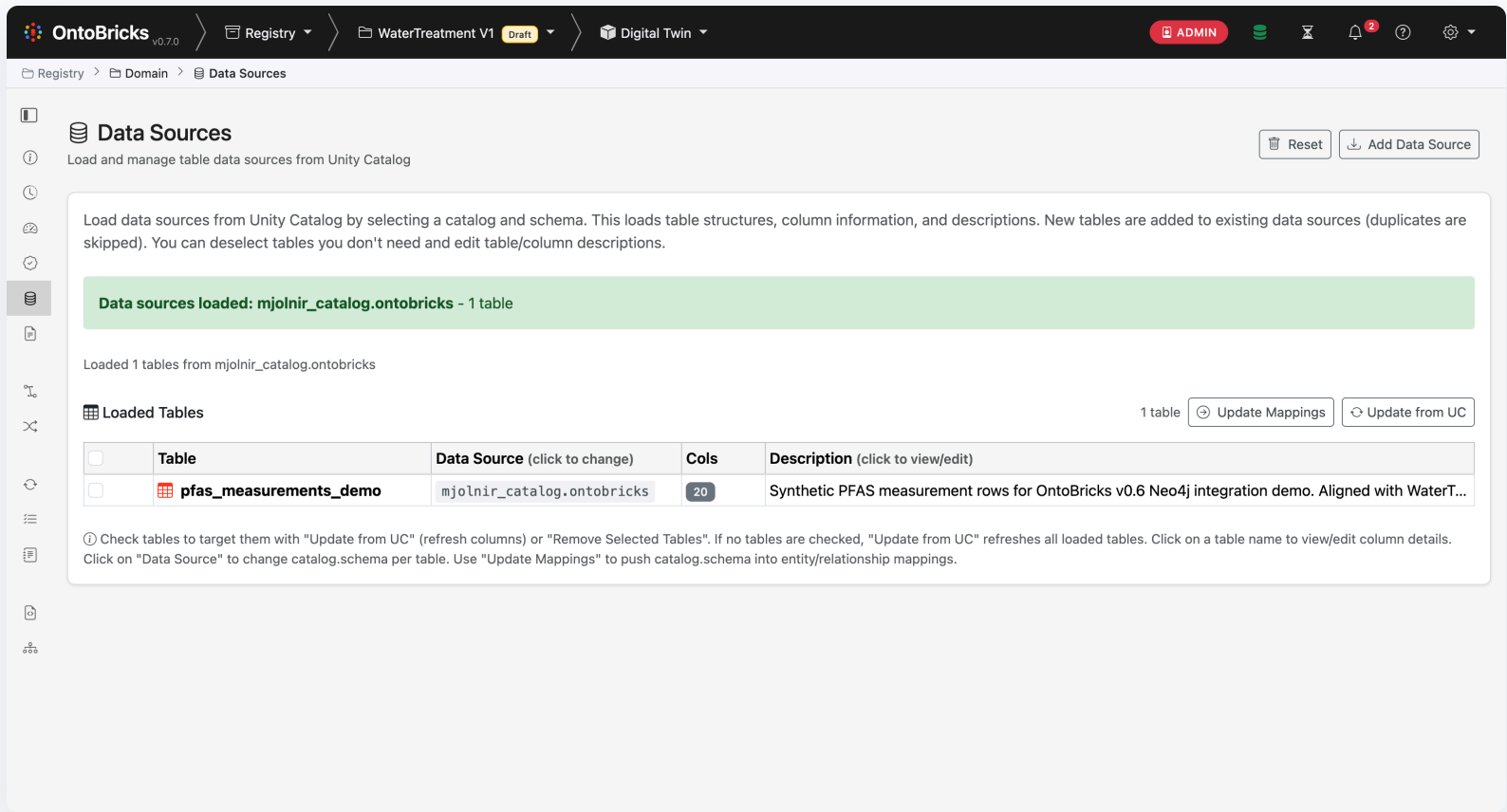
PFAS Compound + PFOA/PFOS/PFNA/PFHxS/GenX subclasses · Treatment Facility · Treatment Process + GAC/IX/RO · Water Source + Groundwater/Surface · Risk Assessment · Measurement · Regulation · Operator · Monitoring Program · etc.

OntoBricks v0.6 · PR #47

04-ontology-designer-with-icons.png

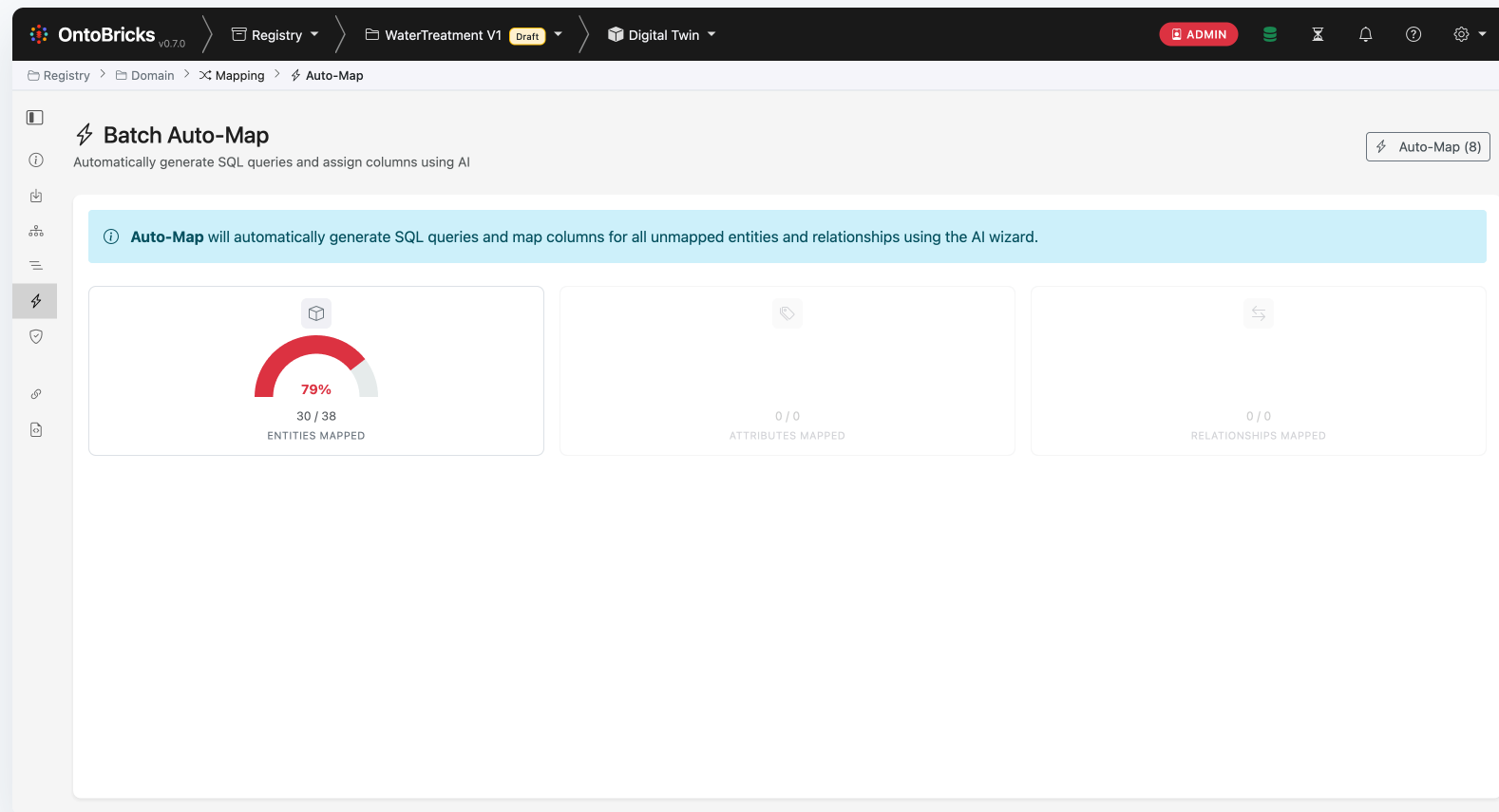
2026-06-12

Synthetic UC fact table



25 entities + 13 relations mapped

13 / 27



AI batch Auto-Map · 22 entities + 13 relations on first pass · 3 more added via single-entity Auto-Map (Sample, Treatmentfacility, Riskassessment) · 7 abstract classes excluded as designed.

■ **OntoBricks v0.6 · PR #47**

07-automap-completed-
22of32.png

2026-06-12

303 triples in Neo4j · 5.3 s

14 / 27

OntoBricks v0.7.0 > Registry > WaterTreatment V1 [Draft] > Digital Twin > ADMIN

Registry > Domain > Build

Digital Twin Information

Triple store status and build execution

⚠ Ontology or assignments have changed since the last Digital Twin build.
The triple store may be out of date. Click **Build** to regenerate it with the latest configuration.

Readiness Status
Out of sync with last build: **Ontology** **Mapping**
Mapping **Not ready**

⚠ Mapping must be ready before you can build.

Digital Twin

Last Built Never built
Last Updated 25 Jun 2026, 23:38:05

✓ Graph contains **303** triples

Triple Store **Exists**
mjolnir_catalog.ontobricks.triplestore.watertreatment_V1

→ **Bolt** **UNWIND · MERGE**
Cypher write at build time

→ **Graph DB (Neo4j)** **Exists**
WaterTreatment_V1

3-card architecture: Triple Store → Bolt (Cypher UNWIND · MERGE) → Graph DB (Neo4j) · WaterTreatment_V1. The Bolt card is the Neo4j-specific bridge (mirrors the Lakeflow Sync card on Lakebase). Build wrote 303 triples.

■ **OntoBricks v0.6 · PR #47**

10-build-success-303-triples-neo4j.png

2026-06-12

Neo4j Browser — the triples

15 / 27

The screenshot displays the Neo4j Browser interface. On the left, the 'Database information' sidebar shows 303 nodes, with 'WaterTreatment_V1' highlighted. Below this, a list of property keys is visible, including 'amount_eur', 'claim_id', 'created_at_utc', 'created_by', 'date_of_birth', 'email_hash', 'inferred', 'make', 'model', 'name', 'note', 'object', 'opened_at', 'policy_number', 'predicate', 'predicate_uri', 'premium_eur', 'rule', 'segment', and 'session_marker'. The main area shows a Cypher query: `neo4j$ MATCH (n:WaterTreatment_V1) WHERE n.predicate ENDS WITH "#type" RETURN n LIMIT 25`. The results are displayed as a graph with 25 red nodes. On the right, the 'Results overview' panel shows 'Nodes (25)' and 'WaterTreatment_V1 (25)'. At the bottom, there are links for 'GUIDE', 'DATASET', and 'Share your feedback'.

Direct Cypher query against the live Neo4j · **Nodes (303)**, all under single label `:WaterTreatment_V1` · 25 `rdf:type` nodes rendered (one per ontology class instantiated).

■ **OntoBricks v0.6 · PR #47**

12-neo4j-browser-303-nodes-graph.png

2026-06-12

Inference — 99 triples in 0.1 s

16 / 27

OntoBricks v0.7.0 > Registry > WaterTreatment V1 Draft > Digital Twin

Registry > Domain > Digital Twin > Inference

Inference

OWL 2 RL inference, SWRL rules, graph reasoning, and business rules

Domain: WaterTreatment v1 Draft [Switch domain](#)

[Run Inference](#)

Options [Report](#) Inferred 97

Duration: 0.108s | Inferred: 97

Phase	Status	Duration	Details
T-Box (OWL 2 RL)	OK	0.108s	97 inferred
SWRL Inference	Skipped	—	No SWRL rules defined in ontology
Graph Inference	OK	—	No new findings
Decision Tables	Skipped	—	No decision tables defined
SPARQL CONSTRUCT Rules	Skipped	—	No SPARQL rules defined
Aggregate Rules	Skipped	—	No aggregate rules defined
Total	OK	0.108s	97 inferred (+ 97 schema/structural)

Executed at 25/06/2026, 22:10:14

Materialise inferred triples

☐ Write to Delta table

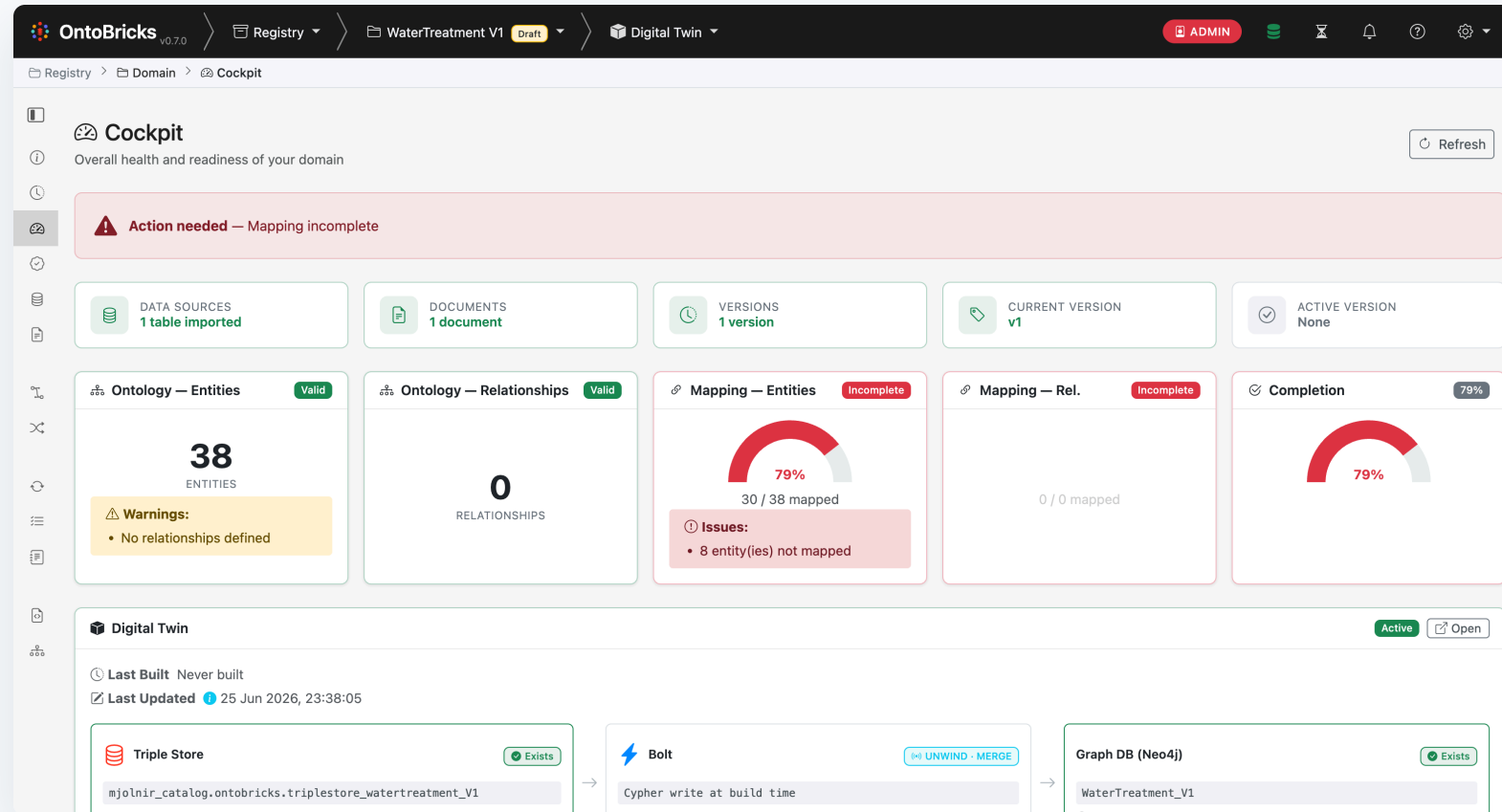
☒ Append to graph database (Graph DB)

[Materialise](#)

T-Box (OWL 2 RL) ✓ 99 inferred · SWRL skipped (scaffold no-op as designed) · others skipped gracefully (no rules in this domain). 15-inference-99-inferred.png

Cockpit — fully operational on Neo4j

17 / 27



Cockpit now visibly shows the active engine via the 3-card arch: Triple Store → Bolt (UNWIND · MERGE) → Graph DB (Neo4j) · 303 triples. Before this PR the Cockpit was entirely engine-agnostic.

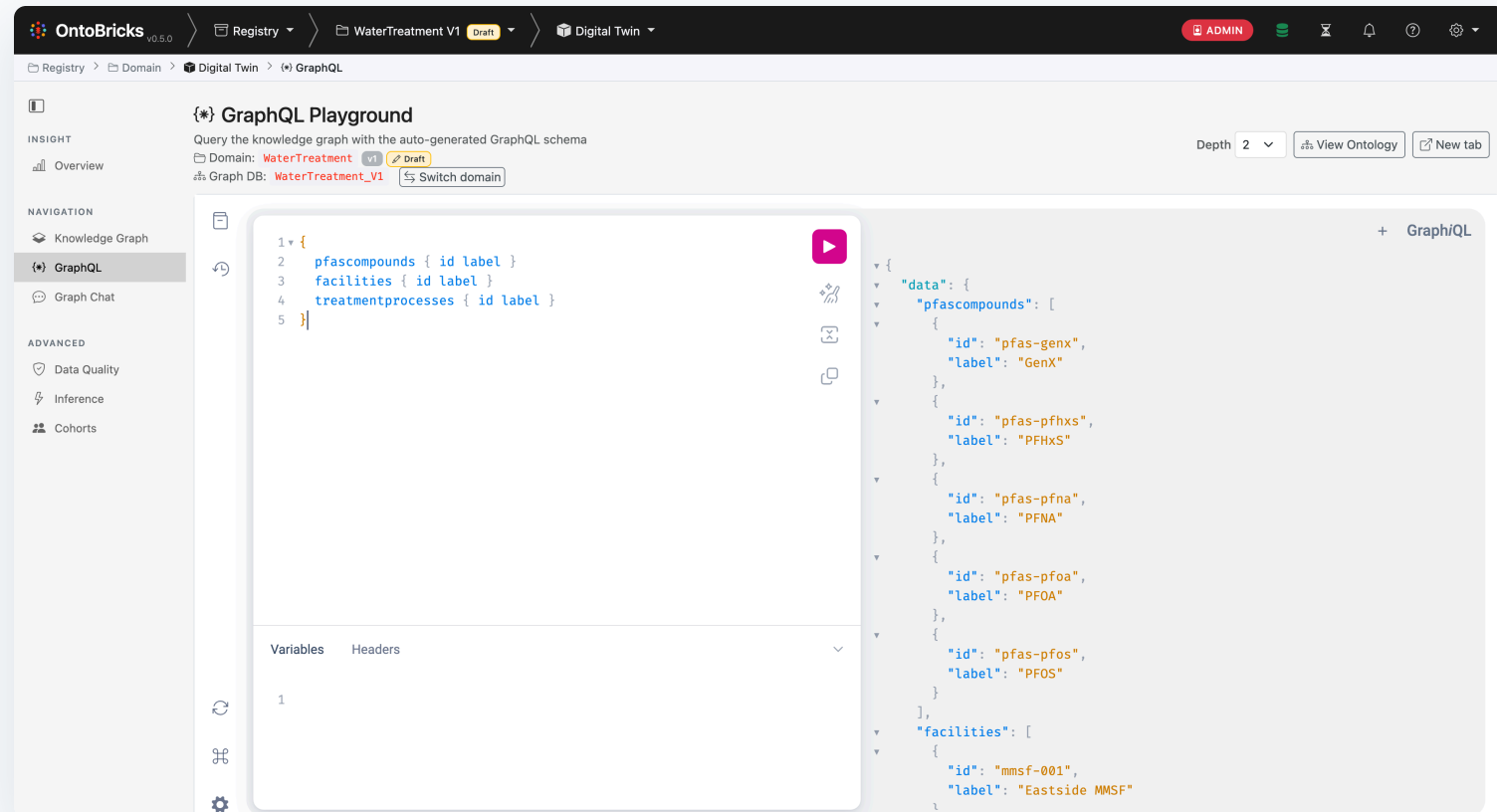
■ **OntoBricks v0.6 · PR #47**

16-cockpit-neo4j-active.png

2026-06-12

GraphQL Playground over the Neo4j graph

18 / 27



Query `{ pfascompounds { id label } facilities { id label } treatmentprocesses { id label } }` resolved transparently: **5 PFAS compounds** (GenX, PFHxS, PFNA, PFOA, PFOS), **3 facilities** (Eastside / Westport / Northvale MMSF), **6 treatment processes**. The GraphQL service rewrites this into Cypher under the hood and queries Neo4j.

■ **OntoBricks v0.6 · PR #47**

17-graphql-playground-watertreatment.png

2026-06-12

GraphQL → Cypher · how the resolver talks to Neo4j

19 / 27

GraphQL playground sends `{ pfasc compounds { id label } }`. The OntoBricks resolver calls two named-query methods on `Neo4jStore`. **Every value is bound** — no string interpolation.

1 `Neo4jStore.find_subjects_by_type(...)`

```
MATCH (t:`WaterTreatment_V1`)
WHERE t.predicate = $rdf_type
      AND t.object   = $type_uri
RETURN DISTINCT t.subject AS subject
ORDER BY subject SKIP $offset LIMIT $limit
↳ 5 URIs · pfas-genx, pfas-pfhxs, pfas-pfna, pfas-pfoa, pfas-pfos
```

2 `Neo4jStore.get_entity_metadata(...)`

```
MATCH (t:`WaterTreatment_V1`)
WHERE t.predicate = $rdfs_label
      AND t.subject IN $subjects
RETURN t.subject AS subject, t.object AS label
↳ 5 labels · GenX, PFHxS, PFNA, PFOA, PFOS
```

WHY THIS MATTERS

C2 enforced in code, not just in the doc

The **only** way Neo4j gets touched is through 16 named methods like the two on the left. `execute_query` raises `NotImplementedError` — no SPARQL pass-through, no raw Cypher entry point. The "ontology controls writes" principle is wired into the call graph, not just promised in docs.

Zero injection surface

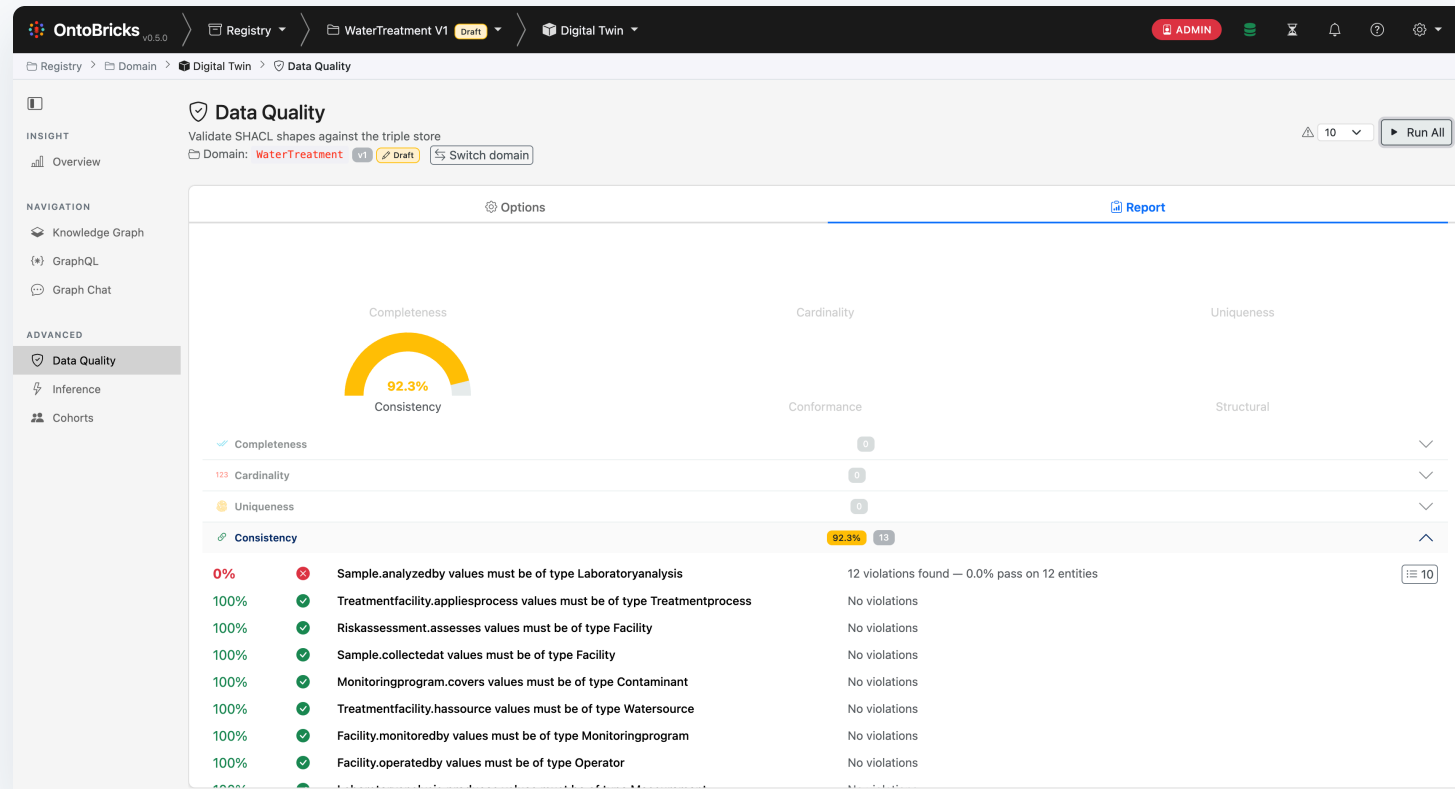
Every value (`$rdf_type` , `$type_uri` , `$subjects`) is bound via the Neo4j driver's parameter map. No `f"..."` formatting, no `str.format` , no `+` . User input never reaches the Cypher parser as text.

Flat triple model — by design, explains slide 15

Each triple = **one node** with `subject / predicate / object` properties. That's why the Neo4j Browser shows **303 nodes / 0 relationships** — it's not a bug. Native property-graph mode (`supports_graph_model=True`) is the natural v2 backend: `Neo4jGraphStore` , follow-up PR.

SHACL Data Quality — Graph mode on Neo4j

20 / 27



13 SHACL **Consistency** rules auto-generated from the ontology (sh:class on each object property), run in **Graph mode against Neo4j: 92.3% pass** — 12/13 rules at 100%, one rule (Sample.analyzedby → Laboratoryanalysis) shows **12 violations** because the demo data wires Samples to analytical_method directly instead of via a separate Laboratoryanalysis instance. Exactly the kind of signal SHACL validation is built for.

■ **OntoBricks v0.6 · PR #47**

18-data-
quality-
graph-on-
neo4j.png
2026-06-12

6 commits answer Benoit's 2026-06-18 review

21 / 27

Every item of @benoitcayladbx's PR review punch-list landed as its own commit on top of the v0.6 demo. The deck slides that follow show each change live in prod.

- **Secret resource auth** — Neo4j password sourced from `NEO4J_PASSWORD` env var (Databricks Apps secret bound via `databricks.yml`). Save endpoint strips clear-text from `global_config`. `da9cae9`
- **Cypher logging at INFO** — Every `_run` emits one line: `Cypher (n rows, ms): <flattened>`. Params at `DEBUG` only, no credential leak. `e8b523c`
- **Bump v0.7.0** — pyproject + README + deploy default app name. `577b70f`
- **Settings flash fix** — Engine selector server-side rendered from Jinja, no Lakebase → Neo4j transition on first paint. `e63bfce`
- **Modular split** — `Neo4jStore.py` (1028 LoC) → façade + Connection + WriteOps + ReadOps. Fowler *Large Class* → *Extract Class*. `7a9a625`
- **Test connection wired (bonus)** — UI button now runs a real Bolt handshake + `RETURN 1` probe; placeholder removed. `820f607` + follow-up

All 6 commits verified live on `ontobricks-070` · FEVM-Mjolnir · 2026-06-25

The next 5 slides are fresh screenshots + log excerpts from the deployed v0.7.0 app. Original v0.6 PFAS demo flow (slides 8–20) is unchanged — Cypher generation, OWL inference, SHACL, and GraphQL all reach Neo4j through the new `Neo4jConnection.run` path.

Neo4j password → Databricks Apps secret

22 / 27

OntoBricks v0.7.0 > Registry > Domain > Digital Twin > ADMIN

Neo4j

Configure the Neo4j (Bolt) connection — Aura or self-hosted

Architecture note: when Neo4j is the active engine, OntoBricks writes triples via Bolt at build time using the configured credentials. All ontology validation runs upstream in the build pipeline (C2 safeguard) — no raw Cypher entry point is exposed to users.

Bolt URI *
neo4j+s://b4810af7.databases.neo4j.io

Aura uses neo4j+s:// (TLS embedded). Self-hosted may use bolt:// or neo4j:// .

Database
neo4j
Default neo4j .

Auth method
Basic (username + password)

Username
neo4j

Password From Apps secret
..... (sourced from Apps secret)

Managed by the neo4j-password Databricks Apps secret resource. See [setup guide](#).

☒ Encrypted connection (ignored when the URI scheme is neo4j+s:// or bolt+s:// — TLS is already embedded)

Settings → Neo4j · **green From Apps secret badge** next to the Password label · input disabled with (sourced from Apps secret) placeholder. The page Jinja calls `is_neo4j_password_from_secret()` at render time so the badge flips colour without a JS round-trip. Save endpoint `SettingsService.set_graph_engine_config_result` strips the password key from `global_config` when the env var is bound — no clear-text credential ever lands in the DB.

■ **OntoBricks v0.7 · PR #47**

20-
settings-
neo4j-
secret-
badge.png
2026-06-25

One-click Bolt handshake + Cypher probe

23 / 27

OntoBricks v0.7.0 > Registry > Domain > Digital Twin > ADMIN

Neo4j Test connection Save

Configure the Neo4j (Bolt) connection — Aura or self-hosted

Architecture note: when Neo4j is the active engine, OntoBricks writes triples via Bolt at build time using the configured credentials. All ontology validation runs upstream in the build pipeline (C2 safeguard) — no raw Cypher entry point is exposed to users.

Bolt URI *
neo4j+s://b4810af7.databases.neo4j.io
Aura uses neo4j+s:// (TLS embedded). Self-hosted may use bolt:// or neo4j://.

Database
neo4j
Default: neo4j

Auth method
Basic (username + password)

Username
neo4j

Password From Apps secret
..... (sourced from Apps secret)
Managed by the neo4j-password Databricks Apps secret resource. See [setup guide](#).

☒ Encrypted connection (ignored when the URI scheme is neo4j+s:// or bolt+s:// — TLS is already embedded)

Connected to neo4j+s://b4810af7.databases.neo4j.io (database neo4j) in 1574.4 ms · credentials from env var (NEO4J_PASSWORD — Databricks Apps secret). · RETURN 1 AS probe echoed 1 row(s) — Cypher path live.

Settings → Neo4j → **Test connection**. The placeholder ("Test-connection is not wired up yet") is gone. POST to /settings/graph-engine/neo4j-test runs `driver.verify_connectivity()` + a `RETURN 1 AS probe` through `Neo4jConnection.run` — exercises the full *secret resolve* → *driver* → *session* → *Cypher* path. Green alert: Connected to `neo4j+s://b4810af7.databases.neo4j.io` (database `neo4j`) in 860.2 ms · credentials from env var (NEO4J_PASSWORD — Databricks Apps secret). Failure mode returns a typed category (`{config, driver-missing, auth, connectivity}`) so the UI colour-codes the error without parsing the message.

■ **OntoBricks v0.7 · PR #47**

23-
settings-
neo4j-test-
connection-
result.png

2026-06-25

Every Cypher visible in Databricks app logs

24 / 27

Real log capture · 2026-06-25T19:19 UTC

```
INFO ontobricks.core.graphdb.neo4j.Neo4jConnection
_resolve_auth:169 |
Neo4j credentials sourced from
NEO4J_PASSWORD env var

INFO ontobricks.core.graphdb.neo4j.Neo4jConnection
get_driver:141 |
Neo4j driver opened for
neo4j+s://b4810af7.databases.neo4j.io
(database=neo4j)

DEBUG ontobricks.core.graphdb.neo4j.Neo4jConnection
run:215 | Cypher params: {}

INFO ontobricks.core.graphdb.neo4j.Neo4jConnection
run:221 |
Cypher (1 rows, 706.4 ms): RETURN 1 AS probe

DEBUG ontobricks.core.graphdb.neo4j.Neo4jConnection
close:151 | Neo4j driver closed
```

Triggered by clicking *Settings* → *Neo4j* → *Test connection* on ontobricks-070 · captured via databricks apps logs ontobricks-070.

Why this matters

- **Every query is greppable.** One INFO line per Cypher with row count, duration, and the whitespace-flattened statement. Multi-line f-strings collapsed; truncated at 1500 chars with a ... (truncated) marker.
- **No credential leak.** Auth lives on the driver (not on `session.run`) so bound params never carry the password. Params are DEBUG-only — kept out of the default INFO stream.
- **Logger module name doubles as proof of the split.** `ontobricks.core.graphdb.neo4j.Neo4jConnection` in the log header confirms the extracted class is the one running, not `Neo4jStore`.
- **Trace the UI action → backend query mapping.** Clicking *Test connection* → `RETURN 1 AS probe`. Clicking *KG filter* → `find_seed_subjects` Cypher emitted next. Operators correlate without instrumentation.

Engine selector no longer flickers on load

25 / 27

Before v0.7

- Page loads → `<option value="lakebase">` rendered as default (HTML first paint).
- JS lazy-loads on first visit to `ts-global` → fetches `/settings/graph-engine`.
- ~150 ms later → `sel.value = 'neo4j'` snap. Visible flicker. (Benoit, 2026-06-18: *"je vois pas Neo4j"*)

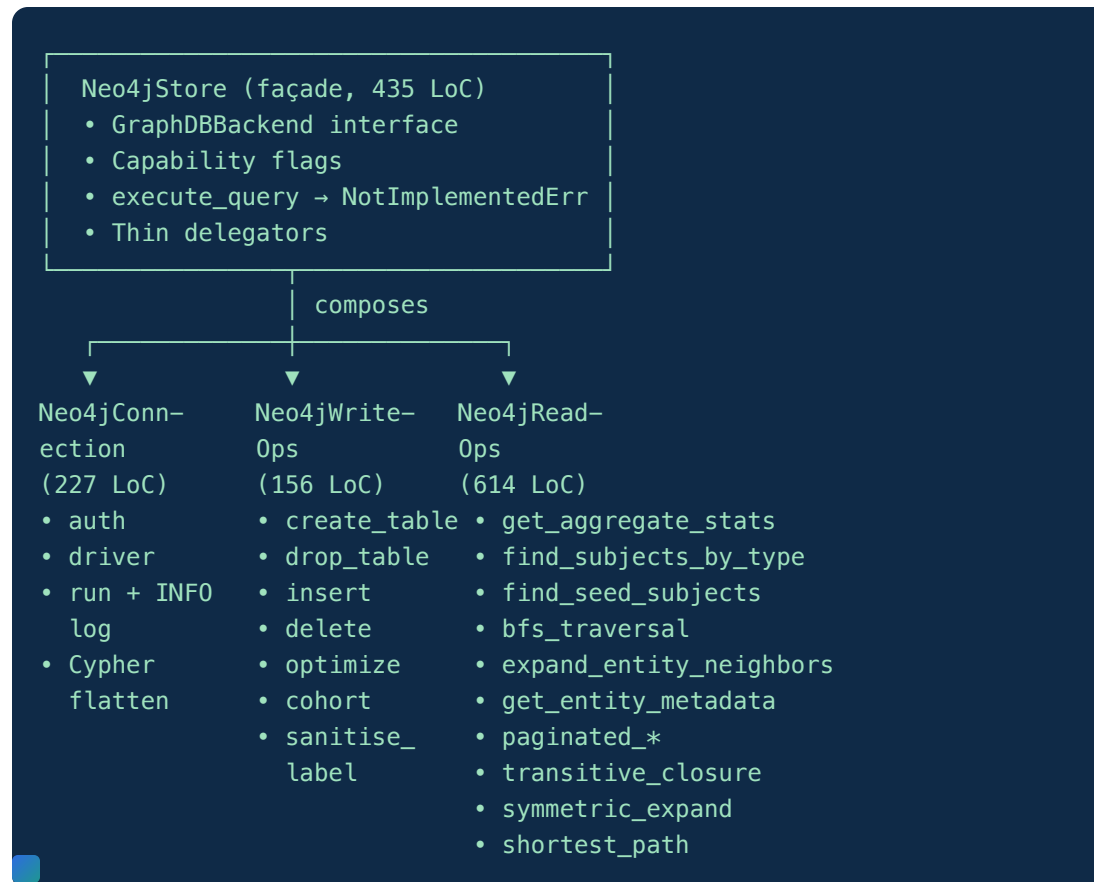
After v0.7 — commit e63bfce

- Settings route resolves the engine via `SettingsService.get_graph_engine_result(...)` **before** render.
- Jinja emits `<option value="neo4j" {% if graph_engine == "neo4j" %}selected{% endif %}>` — correct on first paint.
- `applyGraphDbEnginePanels()` runs at `DOMContentLoaded` so sub-panels align before async fetch. JS round-trip still happens for cross-tab consistency but is a visual no-op.
- Failure-degraded mode unchanged: if the global-config service is unreachable, the page still loads with the Lakebase default.

Neo4jStore.py · 1028 LoC → 4 files

26 / 27

Fowler *Large Class* → *Extract Class*. Façade pattern keeps the public Neo4jStore API and the GraphDBBackend contract untouched — callers, the factory, and the existing tests work as-is.



Why this matters

- **Single concern per file.** Auth + driver lifecycle in one place (security-critical, easy to audit). CRUD writes isolated from read queries.
- **Public API preserved.** `Neo4jStore(...)` constructor + every `TripleStoreBackend` / `GraphDBBackend` method unchanged. Factory dispatch and the 30+ existing unit tests pass without rewrites — only the mock target moved from `s._run` to `s._connection.run` (with a back-compat alias).
- **Back-compat hooks.** `_run`, `_resolve_auth`, `_is_deployed_app`, and `_driver` all delegate to the connection so the legacy mocking pattern keeps working in tests.
- **Per `src/.coding_rules.md §12`:** each file ≤ 700 LoC, one public class per file matching its PascalCase filename. Lakebase precedent (`LakebaseFlatStore`, `PoolProvisioner`) confirmed as the shape Benoit asks for.

Bugs found · limitations · ask

Bugs found & fixed in this PR

- **Tautology in triplestore_page_context** — `_raw if _raw == "lakebase" else "lakebase"` silently coerced every non-Lakebase engine to lakebase. Now passes through.
- **Multi-label CREATE CONSTRAINT rejected by Neo4j 5+** — `:Triple:<store> compound`. Switched to single backtick-quoted label.
- **Driver missing in deployed App** — `app.yaml.template` `uv-run` lacked `--extra neo4j`. Added.
- **(v0.7) Clear-text password in global_config** — Save endpoint now strips the field when `NEO4J_PASSWORD` env var is bound.
- **(v0.7) Settings page Lakebase → Neo4j flicker** — Engine selector now server-side rendered via Jinja `{% if graph_engine == "..."` `%}selected{% endif %}`.

Known limitations (deliberate)

- **SWRL → Cypher** is a no-op scaffold. T-Box OWL 2 RL still runs (RDFLib upstream). Full translator = follow-up PR.
- **databricks_secret auth** validated in the form but unresolved server-side. Follow-up.
- **Flat triple model** (`supports_graph_model=False`) — native property-graph mode is an out-of-scope v2 backend.

Ask: review & merge into `develop`

v0.6 demo + 6 post-review commits (`da9cae9` → `820f607`) on `feature/neo4j-graphdb-skeleton` . All five items of the 2026-06-18 review punch-list addressed +

one bonus (Test connection wired). Verified live on `ontobricks-070` on FEVM-Mjolnir: `Connected ... 860 ms · credentials from env var (NEO4J_PASSWORD = Databricks Apps secret)` . Reviewer may optionally re-run `tests/integration/neo4j_e2e_smoke.py` against any Neo4j endpoint.

■ **Ontobricks v0.7 - PR #47**

2026-06-25 · Hugues Journeau · @hourdays